V2X Slicing Project: Changes and Next Steps

SDN-Orchestrated Network Slicing for Vehicular 5G (V2X)
Work of 5 November 2026, continuing from Section 9 of that day's handoff notes.

1. Where things stand

Step 12 (orchestrator verification) passed, and Step 2 (experiments) is at the pilot stage. The first pilot suggests that slicing protects critical traffic under congestion. Critical round-trip time stayed close to its idle level, compared with over a second under best effort. The threshold loop added little in that pilot, and that needs a closer look because it is the project's claimed research gap.

Item	Status	Detail
Orchestrator runs and switches connect	**Done**	All 3 switches connect, with no OpenFlow errors or crashes in any logged run after the fix
Full connectivity	**Done, recheck**	90/90 in all three pingalls, but the topology has changed since (RSU range)
Same-RSU traffic passes through meters	**Done**	Platoon counters rose from 0 to 229 packets on `rsu1`
Platoon meter limits traffic	**Done**	8 Mb/s flood limited to about 3.9 Mb/s, same-RSU and cross-RSU
Threshold loop switches on and off	**Done**	5 `PROTECT_ON` and 5 `PROTECT_OFF` events in the last Step 12 run
Coverage-hole fix (`RSU_RANGE = 30`)	**Not tested**	Written, not yet run. Second pilot is ready
Stronger `heavy` scenario	**Not tested**	Written, not yet run. Second pilot is ready
Full experiment repeats	**Not started**	Waiting on the second pilot and one policy decision (Section 8)
Plots and write-up	**Not started**	Depends on the full repeats

2. Problems found and fixed

Five problems came up. The first three are fixed and tested. The last two are fixed in the code but not yet tested.

2.1 RYU's event loop crashed at startup

Symptom	The 22:07 session ran for 36 minutes with no switches registered (`"switches": {}`), a single `START` event, and 526 dead connections on port 6653.
Cause	The orchestrator stored its event history in `self.events`, which overwrote the event queue of the same name in RYU's base class. Log: `'collections.deque' object has no attribute 'get'`.
Fix	Renamed the attribute to `self.event_log`. The `/v2x/events` URL is unchanged.
Status	**Tested.** All 3 switches connect, and the crash no longer appears.

2.2 Vehicle-to-vehicle pings failed across RSUs after moving

Symptom	`pingall` reached only 74, 74 and 81 of 90. Every failure was vehicle-to-vehicle between RSUs, and all vehicle-to-server pings worked.
Cause	The old RSU keeps a departed vehicle in its station list for minutes (one entry was still there about 7.5 minutes in). The best explanation is that its kernel delivers frames for that vehicle locally over Wi-Fi, so they never reach OVS. This was inferred from station lists, not from frame captures.
Evidence	Station dumps matched the failures exactly. At the end of the diagnostic run, the only stale entry left was veh2 on `rsu1`, and the only failures left were veh2 with veh3, veh4 and veh5.
Fix	Added the hostapd settings used in the Saraiva et al. code (`ap_max_inactivity=6`, `skip_inactivity_poll=1`, `max_listen_interval=6`) plus 1-second vehicle beacons. The beacons also let RYU see vehicles during movement.
Status	**Tested.** `pingall` reached 89, 90 and 90 of 90, and 4 `HANDOVER` events appeared where there had been none.

2.3 Same-RSU traffic bypassed its slice meter

Symptom	A 1 Mb/s platoon flow between two vehicles on one RSU delivered 260 of 260 packets, yet the platoon rule counters stayed at 0 on every switch.
Cause	The RSU's kernel delivered the frames itself, so they never reached OVS. The meter could not see or limit them.
Fix	Set `client_isolation=True` in the topology. The controller now finds each RSU's Wi-Fi port and sends same-RSU traffic back out through it using `OFPP_IN_PORT`, including broadcasts such as ARP. The `STALE` check now applies to wired ports only.
Status	**Tested.** 229 packets counted on `rsu1` and 0 elsewhere, as expected. An 8 Mb/s flood was limited to 3.96 Mb/s same-RSU and 3.92 Mb/s cross-RSU. `pingall` reached 90/90 in all three runs.

2.4 Traffic gaps of 6 to 14 seconds during handover

Symptom	In the first pilot, flows stopped for 6 to 14 seconds while vehicles moved, with loss of 24% to 75%. The longest gaps were nearly identical in the static and orchestrated runs and similar under best effort.
Likely cause	Mininet-WiFi disconnects a vehicle once it is out of range. The gaps suggest a coverage hole between the two RSUs, which are 40 m apart, so this would not be a slicing effect. The actual range has not been measured.
Fix	Added `RSU_RANGE = 30`, which should give 20 m of overlap.
Status	**Not tested.** Setting `range` may not change the simulated signal strength. If it does not, vehicles at the edge could lose packets or fail to associate.

2.5 The `heavy` scenario was too light to test the loop

Symptom	The backhaul never overflowed (about 18.5 Mb/s of 20 Mb/s under static slicing). `PROTECT_OFF` also had no time to fire before the trial ended.
Fix	Raised critical traffic to about 8.5 Mb/s on the wire (URLLC-far 570 packets/s, platoon 450 packets/s, 1000 bytes each). With eMBB at up to 12 Mb/s, the total can exceed 20 Mb/s. Added 5 seconds at the end of each parked trial.
Status	**Not tested.** This load was chosen so the loop has a case where it could matter. The write-up should say so and report the result even if it shows no benefit.

3. Files changed or added

File	What changed	Tested
`controller/ v2x_orchestrator.py`	Renamed `self.events` to `self.event_log`. Added Wi-Fi port detection and same-port forwarding. Limited `STALE` to wired ports. Added the `V2X_DYNAMIC=0` switch, which turns the threshold loop off.	Yes
`v2x_topology_v2.py`	Added Saraiva hostapd settings, 1-second beacons to 10.0.0.30, `client_isolation=True`, and the `V2X_PLOT=0` option that skips the live plot.	Yes
	Added `RSU_RANGE = 30`.	No
`experiments/ udp_probe.py` (new)	Per-slice UDP probe measuring RTT, jitter, loss and delivery gaps. It is needed because `ping` only travels on the best-effort slice.	Yes
`experiments/ analyze.py` (new)	Turns the logs into `trials.csv` and `summary.md`, with mean and standard deviation across repeats. Only one repeat exists so far, so the deviation calculation has not been exercised.	Yes
`scripts/ run_step2_host.sh`, `run_step2_inner.sh` (new)	Run each scenario under each setup, with setups interleaved in each repeat. The heavier load, longer ending and scenario selection have not been run.	Partly
`scripts/ verify_step12_host.sh`, `verify_step12_inner.sh` (new)	Run the Step 12 checks with one sudo password: `pingall`, station dumps, platoon counters, meter limits, and meter, flow and REST output.	Yes

The project folder is not a git repository, so none of these changes are versioned.

4. Step 12 results, run by run

Run	pingall (of 90)	`HANDOVER`	Same-RSU platoon counter
Before the event-queue fix	Not measured	0	No switches connected
Run 1: event-queue fix	74, 74, 81	0	Not measured
Run 2: added station dumps	74, 74, 81, 84	0	Not measured
Run 3: Saraiva settings and beacons	89, 90, 90	4	0 (bypass confirmed)
Run 4: client isolation and same-port forwarding	**90, 90, 90**	**4**	**229 (bypass fixed)**

The first `pingall` figure in each run is the warm-up. Logs are in `logs/step12_run1` to `logs/step12_run4`.

Other Step 12 checks: 5 meters and 8 slice rules plus the default rule were confirmed in the switch dumps from run 1, and the code that installs them has not changed. The REST `GET` routes respond. The REST `PUT` route that resizes a slice has not been tested.

5. How the literature survey was used

The survey PDF itself does not cover the technical bugs. The most useful source was the code repository it links for Saraiva et al. (`github.com/saraivacode/framework_its_sdn`), which supplied the hostapd settings.

Source	What it shaped
Saraiva et al., IEEE T-ITS 2021 (paper 7)	RSU hostapd settings and the idea of beacon traffic. The three-way comparison: best effort, QoS only and framework. The metrics PDR, throughput and RTT.
Shah et al., IEEE TCE 2025 (paper 2)	Measuring safety traffic during handover, not only while vehicles are parked.
Ahmad, Gul & Derawi, Sensors 2026 (paper 1)	Supports four slices and the focus on infotainment congesting critical traffic.
Survey's combined research gap	The threshold loop, benchmarked on latency, jitter, throughput and loss. This is why the static setup exists: only that comparison shows what the loop adds.

Limits: the Saraiva settings came from reading its code, not the paper's full text, so check the paper before citing them as its method. The beacon script itself (`carcont2.sh`) was not read, so the 1-second ping beacon is this project's own version. The survey says papers 3, 4 and 5 were not read in full, so they were not relied on here.

6. Step 2 experiment design

Three setups

Setup	Controller settings	Meaning
`best_effort`	`V2X_SLICING=0`	Same forwarding, no meters
`static`	`V2X_SLICING=1 V2X_DYNAMIC=0`	Meters at fixed budgets, threshold loop off
`orchestrated`	`V2X_SLICING=1 V2X_DYNAMIC=1`	Meters plus threshold loop

Traffic in each scenario

Flow	Path	Port	`congestion`	`heavy`
URLLC edge	veh4 to `mec_rsu1` (same RSU)	UDP 5000	200 B, 100/s	200 B, 100/s
URLLC far	veh1 to `mec_rsu1` (across backhaul)	UDP 5000	200 B, 100/s	1000 B, 570/s
Platoon	veh2 to veh3 (across backhaul)	UDP 9000	1000 B, 100/s	1000 B, 450/s
mMTC	veh4 to telemetry	UDP 1883	200 B, 10/s	200 B, 10/s
eMBB flood	infotain to veh5, 4 TCP streams	TCP 8080	30 s	30 s

Parked timeline: 25 s settling, then 15 s idle, 30 s flood and 15 s recovery. Results skip the first 5 s of the flood. The `handover` scenario runs while vehicles move, for 30 s: URLLC far (veh1 to `mec_rsu1`), URLLC edge (veh3 to `mec_rsu1`), platoon (veh2 to veh4, 1000 B at 100/s) and the eMBB flood. In that scenario the "edge" and "far" labels describe where each vehicle ends up.

Metrics

For each flow: RTT (mean, 95th and 99th percentile), jitter, loss (PDR is 100% minus loss), throughput and the longest delivery gap. The controller log adds `PROTECT_ON`, `PROTECT_OFF` and `HANDOVER` events. Replies carry only a 12-byte header, because both directions share one meter and full-size replies would use up the budget twice.

7. First pilot results

These come from one run per setup, so they are indicative only. Use them to judge the test design, not as findings. All 9 trials completed, with no controller errors logged.

Parked vehicles, `congestion`, during the flood

Measure	`best_effort`	`static`	`orchestrated`
URLLC edge, mean RTT	688 ms	2.1 ms	1.5 ms
URLLC far, mean RTT	664 ms	11.2 ms	11.0 ms
URLLC far, loss	7.9%	0%	0%
Platoon, mean RTT	1391 ms	12.1 ms	11.5 ms
Platoon, 99th percentile RTT	2642 ms	16.0 ms	15.0 ms
Platoon, jitter	3.94 ms	1.46 ms	0.97 ms
Platoon, loss	5.7%	0%	0%
mMTC, mean RTT	1367 ms	37.2 ms	36.3 ms
eMBB throughput	17.3 Mb/s	11.3 Mb/s	5.6 Mb/s

Idle reference, across all setups: URLLC edge 0.9 ms, URLLC far and platoon 11 ms, mMTC 35.5 ms.

Parked vehicles, old `heavy` load, during the flood

Measure	`best_effort`	`static`	`orchestrated`
Platoon, mean RTT	561 ms	13.7 ms	11.5 ms
Platoon, 99th percentile RTT	813 ms	24.4 ms	14.8 ms
Platoon, loss	1.7%	0%	0%
eMBB throughput	12.1 Mb/s	11.3 Mb/s	5.6 Mb/s

What the pilot suggests

- **Slicing protects critical traffic.** Under both slicing setups, critical RTT stayed close to idle, and critical loss was 0%.
- **Most of the benefit comes from the static meters.** In `congestion`, the loop improved mean RTT by under 1 ms and halved eMBB throughput. In the old `heavy` load, it improved mean platoon RTT by about 2 ms and 99th percentile by about 10 ms, again at half the eMBB throughput.
- **The eMBB limit works.** eMBB throughput stayed under its limit in the parked runs: 11.3 of 12 Mb/s under static, and 5.6 of 6 Mb/s under orchestrated.
- **Wi-Fi is sometimes a bottleneck too.** URLLC edge does not cross the backhaul, yet it reached 688 ms with 17.3 Mb/s of eMBB. The best explanation is that the RSU's Wi-Fi queue also saturates. In the `heavy` best-effort run at 12.1 Mb/s it stayed at 1.7 ms. So best-effort delay is not purely a backhaul effect.
- **Do not use the first pilot's `handover` results.** They reflect the suspected coverage hole.

8. What needs to be done next

Step 1: Run the second pilot (about 8 minutes)

```
sudo bash ~/v2x-project/scripts/run_step2_host.sh 1 "heavy handover"
```

Check that:

- Handover gaps fall well below the 6 to 14 seconds seen so far, and vehicles still change RSU (4 `HANDOVER` events).
- `heavy` under `static` now shows backhaul pressure, and `PROTECT_OFF` fires before each trial ends.

If the gaps remain or vehicles stop associating, `range` did not behave as expected. In that case, try the propagation exponent next (currently `exp=5`; the Saraiva code uses 2.8), and measure the real coverage distance first.

Step 2: Rerun the Step 12 check

The topology has changed since Step 12 passed, so connectivity needs confirming. This also re-exercises the meter and handover checks. It needs the plot display enabled first.

```
DISPLAY=:1 xhost +local:docker

DISPLAY=:1 sudo --preserve-env=DISPLAY bash ~/v2x-project/scripts/verify_step12_host.sh
```

Expect 90/90 after warm-up, 4 `HANDOVER` events, and same-RSU platoon counters above 0. Note that the script overwrites `logs/step12`, so earlier runs are kept in the numbered folders.

Step 3: Decide the threshold policy

This is the main open decision. The loop currently halves eMBB whenever critical traffic reaches 500 kb/s on a switch, even if 12 Mb/s of eMBB would still fit. The options are:

Option	For	Against
Keep the current fixed cut	Simple, already tested, and easy to explain	Gives up half of eMBB throughput, possibly for little benefit
Capacity-aware cut (eMBB budget = link capacity minus critical load minus a margin)	Only cuts eMBB when needed, which may make a stronger contribution	New code and more tuning. All setups need rerunning afterwards

Base this on the new `heavy` result. If the fixed cut shows a clear benefit when the backhaul overflows, keeping it and explaining the trade-off is reasonable. If not, the capacity-aware cut is worth trying. Decide this before the full repeats, because changing it later would mean rerunning them.

Step 4: Finish the remaining functional tests

- REST `PUT` resize, confirming the meter changes on the switches and that traffic follows.
- URLLC, mMTC and best-effort meter limits under an overload. So far only platoon has been directly tested with a flood, and eMBB has only been seen staying under its limit.

Step 5: Run the full experiments (about 70 minutes for 5 repeats)

Keep the pilot logs first, because the runner overwrites matching trial folders:

```
mv ~/v2x-project/logs/step2 ~/v2x-project/logs/step2_pilot2

sudo bash ~/v2x-project/scripts/run_step2_host.sh 5

python3 ~/v2x-project/experiments/analyze.py
```

Stop other containers and heavy programs on the laptop during the run. Five repeats is a practical starting point, not a calculated sample size, so raise it if the spread between repeats is large.

Step 6: Analyse and write up

- Plot RTT distributions and throughput by setup, with error bars across repeats.
- State the limitations: beacons on the best-effort slice, the chosen `heavy` load, Wi-Fi as a second bottleneck, no nearest-MEC steering after handover, and scripted mobility in place of SUMO.
- Pull the full text of the Saraiva paper before citing it, and of papers 3, 4 and 5 before using them, as the survey itself advises.

Housekeeping

- Update the handoff notes. Section 4 is missing the topology changes. Section 6 blames stale MAC learning for the 74/90 result, which does not fit the evidence. Section 9 still lists the orchestrator as unverified.
- Initialise a git repository, so that the exact code behind each result can be identified.
- Remove the leftover root-owned `mn170_rsu1-wlan1.apconf` and `mn170_rsu2-wlan1.apconf` from the project folder. This needs `sudo`.
- Optional items from the handoff: write Environment Setup Guide Part 3, suppress the harmless HTB quantum warnings, and add nearest-MEC steering after handover as future work.

9. Where to find everything

Path (under `~/v2x-project`)	Contents
`logs/step12_run1` to `step12_run4`	Step 12 logs: RYU, topology, OVS and Mininet cleanup
`logs/step2_pilot1/trials.csv`	First pilot, one row per flow, window and trial
`logs/step2_pilot1/summary.md`	First pilot, all summary tables
`logs/step2_pilot1/<scenario>/<setup>/r1`	Raw probe CSVs, eMBB JSON, controller events and meter statistics
`docs/V2X_Progress_Report.pdf`	This report. Its source is `docs/report_src/V2X_Progress_Report.html`